

구조를 먼저 분석하고 코드를 쓰는 백엔드 개발자

정명진

CONTACT

☎ phone : 010-2509 -8557 | ✉ Mail : bnm070289@gmail.com

JEONG MYOUNGJIN

GitHub : <https://github.com/jmj010702>

01. 자기소개

- 프로필 · 기술 스택 (P.03)
-

02. PEOPLECORE 피플코어 (B2B HR 인사 시스템)

- 프로젝트 소개 및 주요 역할 (p.04)
 - 프로젝트 성과 (p.05)
 - 트러블 슈팅 및 회고 (p.06)
-

03. POCHAON 포차온 (실시간 테이블 주문 POS 서비스)

- 프로젝트 소개 및 주요 역할 (p.07)
 - 프로젝트 성과 (p.08)
 - 트러블 슈팅 및 회고 (p.09)
-



정명진

JEONG MYUNGJIN

📅 2001.07.02 (만 25세)

☎ 010-2509-8557

✉ bnm070289@gmail.com

📍 서울, 경기 거주

📍 육군 병장 만기전역

학력 및 교육 사항

2025.11 - 2026.05 한화 SW CAMP 백엔드 과정 수료

2020.02 - 2026.02 나사렛대학교 IT융합학부 졸업

자격사항 및 수상내역

2026.03 PCCE Lv3

2026.01 정보처리 기사 필기 취득

2020.03 운전면허 1종

기타 활동

2025.09 교내 캡스톤 학술제 - ONEROOM RECIPE

2024.09 교내 캡스톤 학술제 - AUTO DIALOG

2023.09 교내 캡스톤 학술제 - AUTO AIR TICKET

기술 스택

언어 Java · Python · JS/TS

프레임워크 Spring Boot · Spring Batch · JPA · QueryDSL · FastAPI · VUE · REACT

DB MySQL · Redis · Elasticsearch · Oracle · PostgreSQL · MinIO

메시징 Kafka · WebSocket · Debezium · SSE

인프라 AWS EKS · ECR · S3 · Docker · GitHub Actions · GitHub Packages · EC2

AI React · Claude · Ollma · EXAONE · RAG

활용 도구

개발도구 IntelliJ IDEA · VS Code · SourceTree

협업도구 Notion · GitHub · Discord · Figma

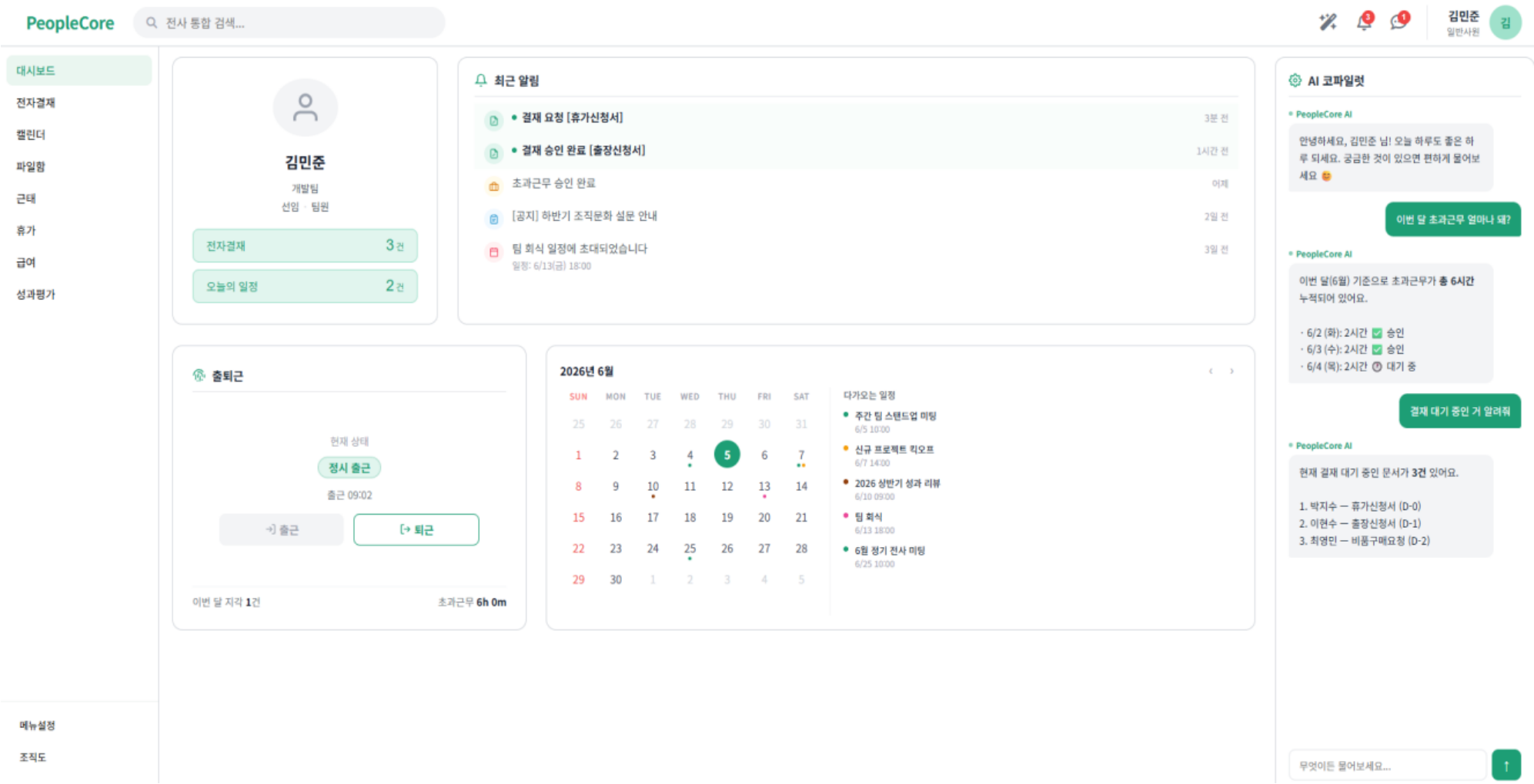
핵심 역량

- 새로운 기술을 빠르게 습득하고 실무에 적용 가능

- 주도적인 커뮤니케이션으로 문제를 해결

PEOPLECORE 피플코어

- [PEOPLECORE 백엔드 레포](#)
- [PEOPLECORE 프론트 레포](#)



프로젝트 소개

- 기업 인사·결재·근태 업무를 통합한 B2B HR SaaS. 회사별 정책 커스터마이징과 도메인별 마이크로서비스 분리를 통한 독립 배포·장애 격리 구조로 설계

핵심 기여

- 전자결재 도메인 설계·동시성 제어
결재 위임·상태 변경 구조를 설계하고, State Pattern과 낙관적 락을 적용해 복잡한 결재 상태와 동시성 충돌을 제어
- 대규모 근태 데이터 처리 최적화
Quartz 기반 배치 구조로 전환하고 MySQL 월별 파티셔닝 및 N+1 개선을 적용해 대량 근태 데이터 처리 구조 최적화
- LLM 기반 AI 코파일럿 설계·구현
민감도 기반 EXAONE·Claude 이중 라우팅과 Tool-Use 구조를 구현하고 Prompt Caching을 적용해 LLM 비용 83% 절감
- MSA 기반 서비스 운영 환경 구축
5개 마이크로서비스를 Gateway·Eureka 기반으로 분리하고 AWS EKS 환경에서 CI/CD·HPA를 포함한 배포 및 운영 환경 구축

프로젝트 성과

성과 01 · AI 비용

LLM 비용 83% 절감

Prompt Caching으로 고정 시스템 프롬프트
재사용하여 반복 전송되는 컨텍스트 비용 최적화

입력 토큰 비용 83% 절감 · 응답 품질 유지

성과 02 · 사용자 경험 개선

AI 코파일럿 결재 프로세스 단순화

AI 코파일럿으로 자연어 한 줄 입력만으로
결재 종류 · 양식 · 결재자 선택 과정을 단순화

결재 종류·양식·결재자 3단계 선택 → 자연어 한줄

성과 03 · 조회 성능 최적화

문서함 카운트 쿼리 9개 → 2개

9개 문서함에 중복 적용되던 검색 · 카운트 로직을
QueryDSL 기반 공통 구조로 통합

중복 카운트 쿼리 9개 → 2개 (78% 감소)

성과 04 · 운영 효율 개선

배치 실행 이력 관리 자동화

배치 실행 이력과 로그를 자동 수집·분석하여
실패 원인 파악 과정 개선

실패 원인 파악 15분 → 3분 · 80% 단축

트러블슈팅 & 회고

01 문서번호 채번의 동시성

문제

- 동시 요청 시 동일 문서번호가 중복 발급되는 문제

해결 과정

- 순번 조회 후 저장하는 과정에서 동시 요청이 끼어들 수 있는 거 확인
- 단순 DB UNIQUE 제약만으로는 충돌 이후의 처리가 부족하다고 판단, 낙관적 락과 비관적 락 비교
- 충돌 가능성과 재시도 비용을 고려해 락 조합과 재시도 전략 적용

해결

- 비관적 락 + 낙관적 락 + 3회 재시도 + UNIQUE 제약으로 중복 발급 차단

배운 점

- 동시성 문제는 하나의 기술을 적용하는 것보다 충돌 지점과 데이터 정합성을 기준으로 여러 방어 전략을 조합하는 것이 중요하다는 것을 배움

02 EKS 서비스 호출 장애

문제

- 로컬에서는 정상, EKS 환경에서 서비스 호출 실패
- 캐시 미스·신규 ID 조회 시 SERVICE_UNAVAILABLE 발생

해결 과정

- redis 캐시 미스 시에만 장애가 발생해 실제 RestClient 호출 구간 추적
- @LoadBalance가 타 service 모듈을 Spring Cloud LoadBalancer의 Service Id로 해석하고 있는거 확인
- Prod에서는 Eureka 비활성화하여 Service ID에 대한 인스턴스를 찾지 못하고 네트워크 요청 전에 실패하는 구조 확인
- 다른 내부 호출은 eks Dns를 직접 사용하고 있어 환경별 서비스 탐색 방식이 달라야 한다고 판단

해결

- Local은 Eureka, Prod는 eks의 DNS를 사용하도록 환경별 호출 구조를 분리

배운 점

- 실행 환경의 구조까지 고려해 환경별 호출 방식을 설계해야 안정적인 MSA 통신을 구현할 수 있음을 배움

03 EXAONE Tool Use 미지원

문제

- EXAONE 3.5 모델이 Tool Use(function Calling) 미지원
- LLM이 도구를 직접 호출할 수 없어 자동화 프로세스 구현 X

해결 과정

- 모델 공식 문서와 테스트로 Tool Use 미지원 사실 확인
- 모델 교체도 검토했으나 성능·비용·도입 일정 제약이 있어 부적합
- 프롬프트 기반 우회 방식 탐색 → 모델이 특정 형식으로 호출을 유도
- 특정 형식의 마커 프로토콜을 설계해 프롬프트에 정의하고 파싱 로직 구현
- 여러 시나리오 테스트로 안정성 · 정확도 검증

해결

- 마커 기반 Tool Calling 프로토콜 구현 및 적용

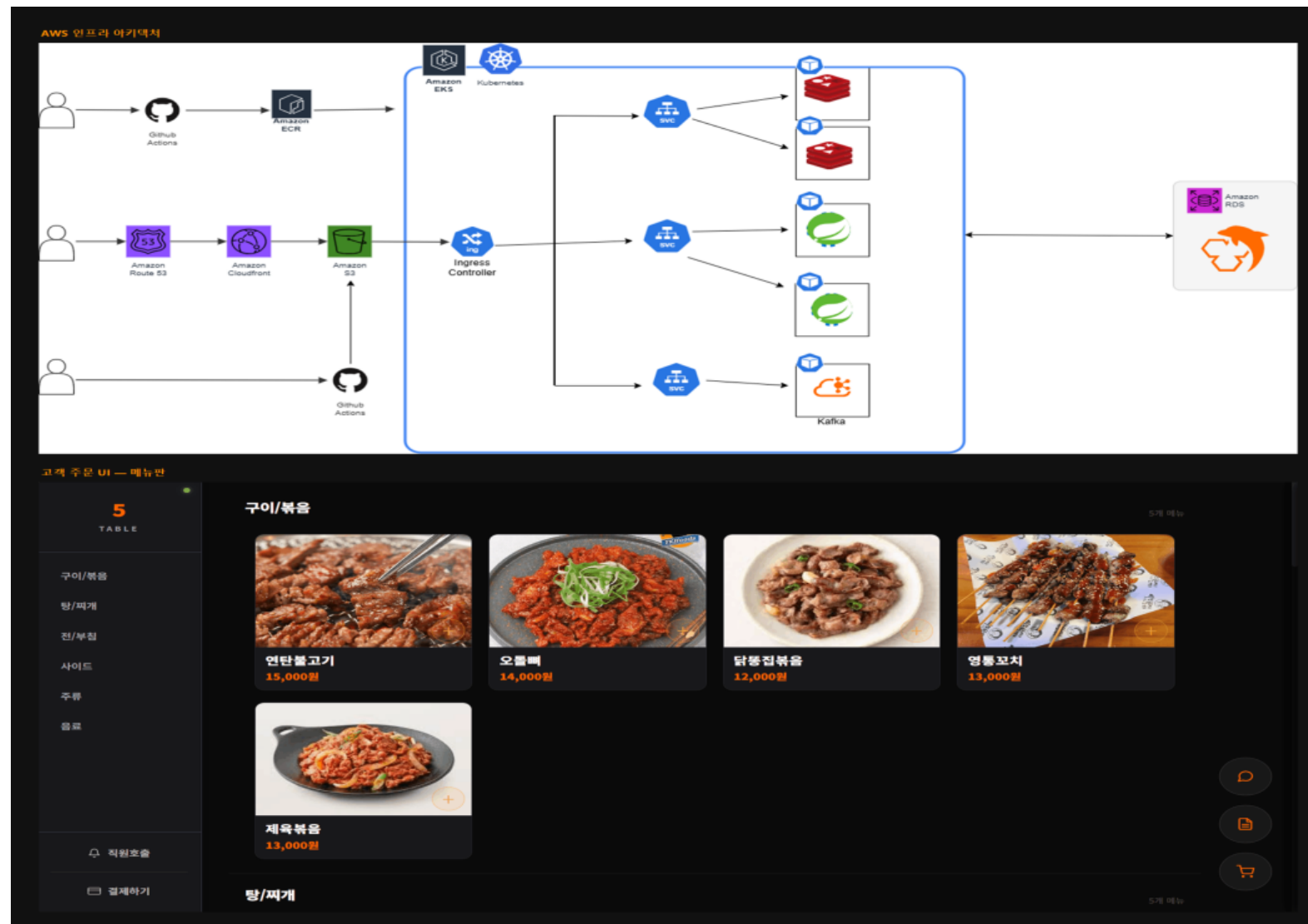
배운 점

- 기술 제약이 있어도 문제 본질을 파악하고 우회 설계를 통해 요구 기능을 안정적으로 구현할 수 있다는 것을 배움

회고 개발하며 기술적인 문제를 해결하는 것만큼, 도메인의 업무 흐름과 규칙으로 이해하는 것이 좋은 설계와 안정적인 서비스 운영에 중요하다는 것을 배우는 프로젝트였습니다

POCHAON 포차온

- POCHAON 백엔드 레포
- POCHAON 프론트 레포



프로젝트 소개

- 매장과 테이블을 잇는 실시간 오더 시스템, 테이블 간 채팅까지 통합한 B2B SaaS Pos 서비스
- 어드민·점주·고객 3역할, 매장별 메뉴·재고 독립 관리 구조

주요 역할

- 주문 처리 - Kafka 이벤트 기반 실시간 테이블 주문 처리
- 동시성 제어 - 비관적 락·Redis 멍등성으로 데이터 정합성 보장
- 실시간 통신 - SSE + WebSocket/Stomp 기반 주문 채팅 구현
- 결제 연동 - Toss Payments API 연동 및 서버 금액 검증
- 정산 자동화 - 스케줄러 기반 매출 마감 배치
- 분산 환경 대응 - Redis Pub/Sub 기반 다중 서버 실시간 메시지 동기화

프로젝트 성과

성과 01 · 안정성 개선

주문 API 동시 처리 안정화

커넥션 풀 부족으로 주문 API 에러가 다수 발생하던 문제를 커넥션 풀 최적화를 통해 해결하고 주문 처리 안정성 확보

성공 건수	에러율	TPS
21건 → 100건	79% → 0%	23.3 → 27.7

성과 02 · 데이터 정확성 확보

재고 차감 정확성 100% 확보

동시 주문 시 재고 차감 누락이 발생하던 문제를 Redis 멍등성 + Kafka 순차처리 + 비관적 락으로 해결

동시 재고 차감 오류	에러율
75건 → 0건	75% → 0%

성과 03 · 실시간 통신 안정화

실시간 메시지 전달 안정화

특정 Pod의 WS 세션으로만 메시지가 전달되어 멀티 Pod 환경에서 메시지 유실이 발생하던 문제를 해결



성과 04 · 성능 최적화

메뉴 추천 조회 구조 개선

매 요청마다 복잡한 집계 쿼리를 수행하던 구조를 사전 계산 방식으로 전환하여 조회 성능 개선



트러블슈팅 & 회고

01 EKS 배포 후 WS/SSE 연결 불안정

문제

- EKS 배포 후 WS가 30초~1분 간격으로 끊김
- SSE 알림도 시간이 지날수록 수신되지 않는 현상 발생

해결 과정

- 재현 및 로그 분석 → Nginx 타임아웃 설정이 원인으로 의심
- WS(STOMP Heartbeat)와 Nginx timeout 값 비교
- SSE 스트리밍이 Nginx 영향을 받는다는 점 확인

해결

- Nginx 설정 최적화
 - proxy-read-timeout 3600s;
 - proxy-send-timeout 3600s;
 - proxy-buffering off;
- WebSocket/SSE 모두 안정적으로 연결 유지

배운 점

- 실시간 통신은 프로그램 설계뿐만 아니라 인프라 세팅이 매우 중요하고 eks의 Nginx의 설정이 서비스 안정성에 영향을 준다는 것을 배움

02 식자재 삭제 시 Cascade 정합성

문제

- 식자재 삭제시 연관 데이터로 인해 FK 제약조건 오류 발생
- 단순 삭제로는 데이터 정합성 유지가 어려움

해결 과정

- 엔티티간 연관관계와 Cascade 적용 범위 추적
- 무작정 CascadeType.all 적용 시 의도하지 않은 데이터 삭제 가능성 검토
- Cascade 대상과 직접 삭제해야 하는 데이터를 분리

해결

- 연관 데이터 선삭제 후 식자재 삭제하도록 삭제 순서 분리

배운 점

- JPA Cascade는 편의성보다 데이터 생명주기와 삭제시 데이터 처리 기준으로 설계해야 한다는 것을 배움

03 결제 승인 데이터 검증

문제

- 외부 결제 승인 요청과 내부 주문 데이터 간 불일치 가능성 발생
- 결제 금액을 중간에서 바꿔 보낼 수 있는 문제

해결 과정

- 결제 승인 과정에서 어떤 값을 신뢰해야 하는지 검토
- 클라이언트 전달값과 서버 주문 데이터를 분리해 검증
- 외부 결제승인 결과까지 확인하는 검증 흐름으로 설계

해결

- 서버 주문 금액을 기준으로 결제 금액 검증 후 승인 처리

배운 점

- 외부 API와 연동할 때는 클라이언트 입력값을 그대로 신뢰하지 않고 서버 기준 데이터로 검증하는 것이 중요함을 배움

회고

다중 서버 환경에서 동시성·분산 환경·외부 API연동까지 고려해야 실제 운영에서 안정적인 서비스를 만들 수 있다는 것을 배운 프로젝트였습니다.